

Python Workshop

Dr.-Ing. Grigory Devadze

July 31, 2025

Einführung in die Programmiersprache Python

- Python ist eine interpretierte, hochentwickelte, universelle Programmiersprache.

- Python ist eine interpretierte, hochentwickelte, universelle Programmiersprache.
- Code oder Quellcode: Die Abfolge von Anweisungen in einem Programm, die von einem Programmierer geschrieben wurden, um bestimmte Aufgaben auszuführen.

- Python ist eine interpretierte, hochentwickelte, universelle Programmiersprache.
- Code oder Quellcode: Die Abfolge von Anweisungen in einem Programm, die von einem Programmierer geschrieben wurden, um bestimmte Aufgaben auszuführen.
- Syntax: Die Menge der zulässigen Strukturen und Befehle, die in einer bestimmten Programmiersprache verwendet werden können. Die Syntax von Python ist so gestaltet, dass sie lesbar und einfach ist.

- Python ist eine interpretierte, hochentwickelte, universelle Programmiersprache.
- Code oder Quellcode: Die Abfolge von Anweisungen in einem Programm, die von einem Programmierer geschrieben wurden, um bestimmte Aufgaben auszuführen.
- Syntax: Die Menge der zulässigen Strukturen und Befehle, die in einer bestimmten Programmiersprache verwendet werden können. Die Syntax von Python ist so gestaltet, dass sie lesbar und einfach ist.
- Ausgabe: Die vom Programm an den Benutzer ausgegebenen Nachrichten, die Text, Zahlen oder andere Informationen enthalten können.

- Konsole: Das Textfeld oder Terminalfenster, in dem Ausgaben angezeigt werden und mit dem der Benutzer mit dem Programm interagieren kann.

- Konsole: Das Textfeld oder Terminalfenster, in dem Ausgaben angezeigt werden und mit dem der Benutzer mit dem Programm interagieren kann.
- Python-Programme können interaktiv (ein Befehl nach dem anderen) oder als Skripte (eine Datei mit einer Befehlsfolge) ausgeführt werden.

- Konsole: Das Textfeld oder Terminalfenster, in dem Ausgaben angezeigt werden und mit dem der Benutzer mit dem Programm interagieren kann.
- Python-Programme können interaktiv (ein Befehl nach dem anderen) oder als Skripte (eine Datei mit einer Befehlsfolge) ausgeführt werden.
- Python wird häufig für Webentwicklung, Datenanalyse, künstliche Intelligenz, wissenschaftliches Rechnen, Automatisierung und mehr verwendet.

- Konsole: Das Textfeld oder Terminalfenster, in dem Ausgaben angezeigt werden und mit dem der Benutzer mit dem Programm interagieren kann.
- Python-Programme können interaktiv (ein Befehl nach dem anderen) oder als Skripte (eine Datei mit einer Befehlsfolge) ausgeführt werden.
- Python wird häufig für Webentwicklung, Datenanalyse, künstliche Intelligenz, wissenschaftliches Rechnen, Automatisierung und mehr verwendet.
- Die große Standardbibliothek und die aktive Community von Python machen es einfach, Werkzeuge und Ressourcen für fast jede Programmieraufgabe zu finden.

Kurzübersicht über die Editoren

- Es gibt viele Editoren und IDEs (Integrierte Entwicklungsumgebungen) für Python, je nach Vorliebe und Anwendungsfall.

Kurzübersicht über die Editoren

- Es gibt viele Editoren und IDEs (Integrierte Entwicklungsumgebungen) für Python, je nach Vorliebe und Anwendungsfall.
- Beliebte Editoren:

Kurzübersicht über die Editoren

- Es gibt viele Editoren und IDEs (Integrierte Entwicklungsumgebungen) für Python, je nach Vorliebe und Anwendungsfall.
- Beliebte Editoren:
 - VS Code: Sehr beliebt, kostenlos, viele Erweiterungen für Python, Debugging, Git-Integration.

- Es gibt viele Editoren und IDEs (Integrierte Entwicklungsumgebungen) für Python, je nach Vorliebe und Anwendungsfall.
- Beliebte Editoren:
 - VS Code: Sehr beliebt, kostenlos, viele Erweiterungen für Python, Debugging, Git-Integration.
 - PyCharm: Leistungsstarke IDE speziell für Python, Community (kostenlos) und Professional (kommerziell).

- Es gibt viele Editoren und IDEs (Integrierte Entwicklungsumgebungen) für Python, je nach Vorliebe und Anwendungsfall.
- Beliebte Editoren:
 - VS Code: Sehr beliebt, kostenlos, viele Erweiterungen für Python, Debugging, Git-Integration.
 - PyCharm: Leistungsstarke IDE speziell für Python, Community (kostenlos) und Professional (kommerziell).
 - Jupyter Notebook: Ideal für Data Science, interaktive Notebooks, Visualisierung und Dokumentation.

Kurzübersicht über die Editoren

- Es gibt viele Editoren und IDEs (Integrierte Entwicklungsumgebungen) für Python, je nach Vorliebe und Anwendungsfall.
- Beliebte Editoren:
 - VS Code: Sehr beliebt, kostenlos, viele Erweiterungen für Python, Debugging, Git-Integration.
 - PyCharm: Leistungsstarke IDE speziell für Python, Community (kostenlos) und Professional (kommerziell).
 - Jupyter Notebook: Ideal für Data Science, interaktive Notebooks, Visualisierung und Dokumentation.
 - Thonny: Einfache IDE für Einsteiger, besonders für den Unterricht geeignet.

Kurzübersicht über die Editoren

- Es gibt viele Editoren und IDEs (Integrierte Entwicklungsumgebungen) für Python, je nach Vorliebe und Anwendungsfall.
- Beliebte Editoren:
 - VS Code: Sehr beliebt, kostenlos, viele Erweiterungen für Python, Debugging, Git-Integration.
 - PyCharm: Leistungsstarke IDE speziell für Python, Community (kostenlos) und Professional (kommerziell).
 - Jupyter Notebook: Ideal für Data Science, interaktive Notebooks, Visualisierung und Dokumentation.
 - Thonny: Einfache IDE für Einsteiger, besonders für den Unterricht geeignet.
 - Sublime Text, Atom: Leichte Editoren mit Python-Unterstützung.

Kurzübersicht über die Editoren

- Es gibt viele Editoren und IDEs (Integrierte Entwicklungsumgebungen) für Python, je nach Vorliebe und Anwendungsfall.
- Beliebte Editoren:
 - VS Code: Sehr beliebt, kostenlos, viele Erweiterungen für Python, Debugging, Git-Integration.
 - PyCharm: Leistungsstarke IDE speziell für Python, Community (kostenlos) und Professional (kommerziell).
 - Jupyter Notebook: Ideal für Data Science, interaktive Notebooks, Visualisierung und Dokumentation.
 - Thonny: Einfache IDE für Einsteiger, besonders für den Unterricht geeignet.
 - Sublime Text, Atom: Leichte Editoren mit Python-Unterstützung.
 - Vim, Emacs: Mächtige, anpassbare Editoren für Fortgeschrittene.

- Viele Editoren bieten:

- Viele Editoren bieten:
 - Syntax-Highlighting

- Viele Editoren bieten:
 - Syntax-Highlighting
 - Autovervollständigung

- Viele Editoren bieten:
 - Syntax-Highlighting
 - Autovervollständigung
 - Debugging-Tools

- Viele Editoren bieten:
 - Syntax-Highlighting
 - Autovervollständigung
 - Debugging-Tools
 - Integration mit Versionskontrolle (z.B. Git)

- Viele Editoren bieten:
 - Syntax-Highlighting
 - Autovervollständigung
 - Debugging-Tools
 - Integration mit Versionskontrolle (z.B. Git)
 - Erweiterungen/Plugins für zusätzliche Funktionalität

Die Help-Funktion

- Python bietet eine eingebaute Hilfe-Funktion, um Informationen zu Modulen, Funktionen, Klassen und Objekten direkt im Interpreter abzurufen.

```
help(print)
```

```
help(str)
```

```
import math
```

```
help(math.sqrt)
```

Die Help-Funktion

- Python bietet eine eingebaute Hilfe-Funktion, um Informationen zu Modulen, Funktionen, Klassen und Objekten direkt im Interpreter abzurufen.
- Die Funktion 'help()' kann auf jedes Python-Objekt angewendet werden, um dessen Dokumentation anzuzeigen.

```
help(print)
```

```
help(str)
```

```
import math
```

```
help(math.sqrt)
```

Die Help-Funktion

- Python bietet eine eingebaute Hilfe-Funktion, um Informationen zu Modulen, Funktionen, Klassen und Objekten direkt im Interpreter abzurufen.
- Die Funktion 'help()' kann auf jedes Python-Objekt angewendet werden, um dessen Dokumentation anzuzeigen.
- Besonders nützlich beim interaktiven Arbeiten oder zum schnellen Nachschlagen von Syntax und Parametern.

```
help(print)
```

```
help(str)
```

```
import math
```

```
help(math.sqrt)
```

- Im interaktiven Interpreter kann auch einfach 'help()' ohne Argument aufgerufen werden, um ein Hilfemenü zu starten.

```
print(print.__doc__)
```

- Tipp: In Jupyter Notebooks kann ein Fragezeichen nach dem Funktionsnamen verwendet werden, z.B. `print?`, um die Hilfe anzuzeigen

- Im interaktiven Interpreter kann auch einfach 'help()' ohne Argument aufgerufen werden, um ein Hilfemenü zu starten.
- Viele Objekte haben außerdem ein spezielles Attribut `__doc__`, das den Docstring enthält:

```
print(print.__doc__)
```

- Tipp: In Jupyter Notebooks kann ein Fragezeichen nach dem Funktionsnamen verwendet werden, z.B. `print?`, um die Hilfe anzuzeigen

- Viele Sprachen erfordern, dass Sie Ihr Programm kompilieren (übersetzen), damit der Computer es versteht.

```
grigo@Grigorys-MBP ~/gits/seminars/gfu/Python_individuell/src r main ± python3
Python 3.13.3 (main, Apr 8 2025, 13:54:08) [Clang 17.0.0 (clang-1700.0.13.3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Willkommen zum Python Workshop")
Willkommen zum Python Workshop
>>>
```

Kompilieren und Interpretieren

- Viele Sprachen erfordern, dass Sie Ihr Programm kompilieren (übersetzen), damit der Computer es versteht.
- Python wird stattdessen direkt in Maschinenbefehle interpretiert.

```
grigo@Grigorys-MBP ~/gits/seminars/gfu/Python_individuell/src r main ± python3
Python 3.13.3 (main, Apr 8 2025, 13:54:08) [Clang 17.0.0 (clang-1700.0.13.3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Willkommen zum Python Workshop")
Willkommen zum Python Workshop
>>>
```

Unterschied zu Java / C#.NET / C++ / PHP (1)

- Python ist eine interpretierte Sprache, während Java, C#.NET und C++ kompiliert werden
- PHP: interpretiert, aber mit anderer Syntax und Semantik
- Python benötigt keine explizite Typdeklaration (dynamisch typisiert), während Java, C# und C++ statisch typisiert sind.
- Python verwendet Einrückung zur Blockstrukturierung, keine geschweiften Klammern wie in Java, C#, C++ oder PHP.
- Python-Code ist in der Regel kürzer und lesbarer, da viele Sprachkonstrukte und Standardbibliotheken sehr mächtig und einfach zu nutzen sind.

- Python unterstützt mehrere Programmierparadigmen: prozedural, objektorientiert und funktional.

- Python unterstützt mehrere Programmierparadigmen: prozedural, objektorientiert und funktional.
- Speicherverwaltung (Garbage Collection) ist in Python automatisch, ähnlich wie in Java und C#, aber anders als in C++ (manuell).

- Python unterstützt mehrere Programmierparadigmen: prozedural, objektorientiert und funktional.
- Speicherverwaltung (Garbage Collection) ist in Python automatisch, ähnlich wie in Java und C#, aber anders als in C++ (manuell).
- Python ist plattformunabhängig und wird häufig für Skripting, Automatisierung, Data Science und Webentwicklung verwendet.

- Python unterstützt mehrere Programmierparadigmen: prozedural, objektorientiert und funktional.
- Speicherverwaltung (Garbage Collection) ist in Python automatisch, ähnlich wie in Java und C#, aber anders als in C++ (manuell).
- Python ist plattformunabhängig und wird häufig für Skripting, Automatisierung, Data Science und Webentwicklung verwendet.
- Die Community und die Anzahl an verfügbaren Bibliotheken für Python ist sehr groß, was die Entwicklung beschleunigt.

Der Python-Interpreter

- Der Interpreter bietet eine interaktive Umgebung, um mit der Sprache zu experimentieren.

```
>>> 3 + 7
```

```
10
```

```
>>> 3 < 15
```

```
True
```

```
>>> 'print me'
```

```
'print me'
```

```
>>> print('print me')
```

```
print me
```

Der Python-Interpreter

- Der Interpreter bietet eine interaktive Umgebung, um mit der Sprache zu experimentieren.
- Ergebnisse von Ausdrücken werden auf dem Bildschirm ausgegeben.

```
>>> 3 + 7
```

```
10
```

```
>>> 3 < 15
```

```
True
```

```
>>> 'print me'
```

```
'print me'
```

```
>>> print('print me')
```

```
print me
```

- Sie können den Python-Interpreter auf zwei Hauptarten verwenden:

- Sie können den Python-Interpreter auf zwei Hauptarten verwenden:
 - Interaktiver Modus: Geben Sie 'python' oder 'python3' in Ihr Terminal ein und geben Sie Befehle einzeln ein.

- Sie können den Python-Interpreter auf zwei Hauptarten verwenden:
 - Interaktiver Modus: Geben Sie 'python' oder 'python3' in Ihr Terminal ein und geben Sie Befehle einzeln ein.
 - Skriptmodus: Speichern Sie Ihren Code in einer '.py'-Datei und führen Sie ihn mit 'python dateiname.py' aus.

- Sie können den Python-Interpreter auf zwei Hauptarten verwenden:
 - Interaktiver Modus: Geben Sie 'python' oder 'python3' in Ihr Terminal ein und geben Sie Befehle einzeln ein.
 - Skriptmodus: Speichern Sie Ihren Code in einer '.py'-Datei und führen Sie ihn mit 'python dateiname.py' aus.
- Der Interpreter wertet Ausdrücke sofort aus und zeigt deren Ergebnisse an.

- Sie können den Python-Interpreter auf zwei Hauptarten verwenden:
 - Interaktiver Modus: Geben Sie 'python' oder 'python3' in Ihr Terminal ein und geben Sie Befehle einzeln ein.
 - Skriptmodus: Speichern Sie Ihren Code in einer '.py'-Datei und führen Sie ihn mit 'python dateiname.py' aus.
- Der Interpreter wertet Ausdrücke sofort aus und zeigt deren Ergebnisse an.
- Sie können den Interpreter als Taschenrechner, zum Testen oder zum Erkunden von Python-Funktionen verwenden.

```
$ python3
Python 3.10.12 ...
Type "help", "copyright", "credits" or "license" ...
>>> 2 * 5
10
>>> print("Hello, Python!")
Hello, Python!
>>> exit()
```

- Die Eingabeaufforderung >>> zeigt an, dass der Interpreter bereit ist.
- Verwenden Sie `exit()` oder drücken Sie 'Ctrl+D' (auf Unix/macOS) oder 'Ctrl+Z' (auf Windows), um den Interpreter zu verlassen.

- Ausdruck: Ein Datenwert oder eine Reihe von Operationen zur Berechnung eines Wertes.

$1 + 4 * 3 \quad \# \quad 13$

$(1 + 3) * 4 \quad \# \quad 16$

- Ausdruck: Ein Datenwert oder eine Reihe von Operationen zur Berechnung eines Wertes.
- Arithmetische Operatoren: +, -, *, /, %, **

1 + 4 * 3 # 13

(1 + 3) * 4 # 16

- Ausdruck: Ein Datenwert oder eine Reihe von Operationen zur Berechnung eines Wertes.
- Arithmetische Operatoren: +, -, *, /, %, **
- Vorrang: * / % ** haben einen höheren Vorrang als + und -

1 + 4 * 3 # 13

(1 + 3) * 4 # 16

- Ausdruck: Ein Datenwert oder eine Reihe von Operationen zur Berechnung eines Wertes.
- Arithmetische Operatoren: +, -, *, /, %, **
- Vorrang: * / % ** haben einen höheren Vorrang als + und -
- Klammern können verwendet werden, um eine bestimmte Auswertungsreihenfolge zu erzwingen.

1 + 4 * 3 # 13

(1 + 3) * 4 # 16

- Bei der Division von Ganzzahlen mit `/` ist der Quotient ebenfalls eine Ganzzahl (Python 2).

```
35 / 5    # 7
```

```
84 / 10   # 8
```

```
156 / 100 # 1
```

```
43 % 5    # 3
```

- Bei der Division von Ganzzahlen mit / ist der Quotient ebenfalls eine Ganzzahl (Python 2).
- Der % Operator berechnet den Rest einer Ganzzahldivision.

35 / 5 # 7

84 / 10 # 8

156 / 100 # 1

43 % 5 # 3

- Python kann auch mit Gleitkommazahlen (floats) umgehen.

```
15.0 / 2.0  # 7.5
```

```
1 / 2.0     # 0.5
```

- Python kann auch mit Gleitkommazahlen (floats) umgehen.
- Der / Operator liefert ein exaktes Ergebnis (Python 3).

```
15.0 / 2.0 # 7.5
```

```
1 / 2.0     # 0.5
```

- Python kann auch mit Gleitkommazahlen (floats) umgehen.
- Der / Operator liefert ein exaktes Ergebnis (Python 3).
- Wenn Ganzzahlen und Gleitkommazahlen gemischt werden, ist das Ergebnis ein float.

```
15.0 / 2.0  # 7.5
```

```
1 / 2.0     # 0.5
```

- Einige häufig verwendete mathematische Funktionen und Konstanten sind:

- Einige häufig verwendete mathematische Funktionen und Konstanten sind:
 - $\text{abs}(x)$: Absolutwert

- Einige häufig verwendete mathematische Funktionen und Konstanten sind:
 - `abs(x)`: Absolutwert
 - `ceil(x)`: Kleinste ganze Zahl $\geq x$

- Einige häufig verwendete mathematische Funktionen und Konstanten sind:
 - `abs(x)`: Absolutwert
 - `ceil(x)`: Kleinste ganze Zahl $\geq x$
 - `floor(x)`: Größte ganze Zahl $\leq x$

- Einige häufig verwendete mathematische Funktionen und Konstanten sind:
 - $\text{abs}(x)$: Absolutwert
 - $\text{ceil}(x)$: Kleinste ganze Zahl $\geq x$
 - $\text{floor}(x)$: Größte ganze Zahl $\leq x$
 - $\cos(x)$, $\sin(x)$, $\tan(x)$: Trigonometrische Funktionen (x im Bogenmaß)

- Einige häufig verwendete mathematische Funktionen und Konstanten sind:
 - `abs(x)`: Absolutwert
 - `ceil(x)`: Kleinste ganze Zahl $\geq x$
 - `floor(x)`: Größte ganze Zahl $\leq x$
 - `cos(x)`, `sin(x)`, `tan(x)`: Trigonometrische Funktionen (x im Bogenmaß)
 - `sqrt(x)`: Quadratwurzel

- Einige häufig verwendete mathematische Funktionen und Konstanten sind:
 - `abs(x)`: Absolutwert
 - `ceil(x)`: Kleinste ganze Zahl $\geq x$
 - `floor(x)`: Größte ganze Zahl $\leq x$
 - `cos(x)`, `sin(x)`, `tan(x)`: Trigonometrische Funktionen (x im Bogenmaß)
 - `sqrt(x)`: Quadratwurzel
 - `log(x)`: Natürlicher Logarithmus (Basis e)

- Einige häufig verwendete mathematische Funktionen und Konstanten sind:
 - `abs(x)`: Absolutwert
 - `ceil(x)`: Kleinste ganze Zahl $\geq x$
 - `floor(x)`: Größte ganze Zahl $\leq x$
 - `cos(x)`, `sin(x)`, `tan(x)`: Trigonometrische Funktionen (x im Bogenmaß)
 - `sqrt(x)`: Quadratwurzel
 - `log(x)`: Natürlicher Logarithmus (Basis e)
 - `log10(x)`: Logarithmus zur Basis 10

- Einige häufig verwendete mathematische Funktionen und Konstanten sind:
 - `abs(x)`: Absolutwert
 - `ceil(x)`: Kleinste ganze Zahl $\geq x$
 - `floor(x)`: Größte ganze Zahl $\leq x$
 - `cos(x)`, `sin(x)`, `tan(x)`: Trigonometrische Funktionen (x im Bogenmaß)
 - `sqrt(x)`: Quadratwurzel
 - `log(x)`: Natürlicher Logarithmus (Basis e)
 - `log10(x)`: Logarithmus zur Basis 10
 - `pow(x, y)`: x hoch y

- Einige häufig verwendete mathematische Funktionen und Konstanten sind:
 - `abs(x)`: Absolutwert
 - `ceil(x)`: Kleinste ganze Zahl $\geq x$
 - `floor(x)`: Größte ganze Zahl $\leq x$
 - `cos(x)`, `sin(x)`, `tan(x)`: Trigonometrische Funktionen (x im Bogenmaß)
 - `sqrt(x)`: Quadratwurzel
 - `log(x)`: Natürlicher Logarithmus (Basis e)
 - `log10(x)`: Logarithmus zur Basis 10
 - `pow(x, y)`: x hoch y
 - `max(x, y)`, `min(x, y)`: Maximum/Minimum von zwei Werten

- Einige häufig verwendete mathematische Funktionen und Konstanten sind:
 - `abs(x)`: Absolutwert
 - `ceil(x)`: Kleinste ganze Zahl $\geq x$
 - `floor(x)`: Größte ganze Zahl $\leq x$
 - `cos(x)`, `sin(x)`, `tan(x)`: Trigonometrische Funktionen (x im Bogenmaß)
 - `sqrt(x)`: Quadratwurzel
 - `log(x)`: Natürlicher Logarithmus (Basis e)
 - `log10(x)`: Logarithmus zur Basis 10
 - `pow(x, y)`: x hoch y
 - `max(x, y)`, `min(x, y)`: Maximum/Minimum von zwei Werten
 - `round(x)`: Nächste ganze Zahl

- Einige häufig verwendete mathematische Funktionen und Konstanten sind:
 - `abs(x)`: Absolutwert
 - `ceil(x)`: Kleinste ganze Zahl $\geq x$
 - `floor(x)`: Größte ganze Zahl $\leq x$
 - `cos(x)`, `sin(x)`, `tan(x)`: Trigonometrische Funktionen (x im Bogenmaß)
 - `sqrt(x)`: Quadratwurzel
 - `log(x)`: Natürlicher Logarithmus (Basis e)
 - `log10(x)`: Logarithmus zur Basis 10
 - `pow(x, y)`: x hoch y
 - `max(x, y)`, `min(x, y)`: Maximum/Minimum von zwei Werten
 - `round(x)`: Nächste ganze Zahl
 - `e`, `pi`: Mathematische Konstanten

```
from math import *  
abs(-5)          # 5  
ceil(2.3)        # 3.0  
cos(0)           # 1.0  
sqrt(16)         # 4.0  
floor(2.9)       # 2.0  
log(10)          # 2.302585092994046  
log10(100)       # 2.0  
pow(2, 3)        # 8.0  
max(5, 9)        # 9  
min(5, 9)        # 5  
round(2.718)     # 3  
e                # 2.718281828459045  
pi               # 3.141592653589793
```

- Variable: Ein benannter Speicherbereich, der einen Wert speichern kann.

- Variable: Ein benannter Speicherbereich, der einen Wert speichern kann.
- Zuweisungsanweisung: Speichert einen Wert in einer Variablen.

- Variable: Ein benannter Speicherbereich, der einen Wert speichern kann.
- Zuweisungsanweisung: Speichert einen Wert in einer Variablen.
- Variablen in Python sind dynamisch typisiert, d.h. Sie müssen den Typ vor der Zuweisung nicht deklarieren.

- Variable: Ein benannter Speicherbereich, der einen Wert speichern kann.
- Zuweisungsanweisung: Speichert einen Wert in einer Variablen.
- Variablen in Python sind dynamisch typisiert, d.h. Sie müssen den Typ vor der Zuweisung nicht deklarieren.
- Sie können jeder Variablen einen beliebigen Typ zuweisen (Zahl, String, Liste usw.), und der Typ kann sich während der Programmausführung ändern.

- Variable: Ein benannter Speicherbereich, der einen Wert speichern kann.
- Zuweisungsanweisung: Speichert einen Wert in einer Variablen.
- Variablen in Python sind dynamisch typisiert, d.h. Sie müssen den Typ vor der Zuweisung nicht deklarieren.
- Sie können jeder Variablen einen beliebigen Typ zuweisen (Zahl, String, Liste usw.), und der Typ kann sich während der Programmausführung ändern.
- Variablennamen sollten mit einem Buchstaben oder Unterstrich beginnen und können Buchstaben, Zahlen und Unterstriche enthalten.

```
x = 5
gpa = 3.14
name = "Alice"
x = x + 4   # 9
gpa = gpa * 2   # 6.28
x = "hello"    # x ist jetzt ein String
```

- Sie können mehreren Variablen gleichzeitig denselben Wert zuweisen:

```
a = b = c = 0
```

- Sie können auch mehrere Werte gleichzeitig zuweisen (Tuple-Unpacking):

```
x, y, z = 1, 2, 3
```


- print: Gibt Textausgaben auf der Konsole aus.

- print: Gibt Textausgaben auf der Konsole aus.
- Die print-Funktion kann mehrere Elemente ausgeben, getrennt durch Kommas (fügt Leerzeichen ein).

- print: Gibt Textausgaben auf der Konsole aus.
- Die print-Funktion kann mehrere Elemente ausgeben, getrennt durch Kommas (fügt Leerzeichen ein).
- Sie können formatierte Strings (f-Strings, verfügbar ab Python 3.6+) für lesbarere und flexiblere Ausgaben verwenden.

- `print`: Gibt Textausgaben auf der Konsole aus.
- Die `print`-Funktion kann mehrere Elemente ausgeben, getrennt durch Kommas (fügt Leerzeichen ein).
- Sie können formatierte Strings (f-Strings, verfügbar ab Python 3.6+) für lesbarere und flexiblere Ausgaben verwenden.
- Sie können auch das ältere %-Format oder `str.format()` für String-Interpolation verwenden.

```
print("Hello, world!")
age = 45
print("Sie haben", 65 - age, "Jahre bis zur Rente")
# Mit f-Strings (Python 3.6+)
print(f"Sie haben {65 - age} Jahre bis zur Rente")
# Mit str.format()
print("Sie haben {} Jahre bis zur Rente".format(65 - age))
# Mit %-Formatierung
print("Sie haben %d Jahre bis zur Rente" % (65 - age))
```

- Standardmäßig fügt print am Ende einen Zeilenumbruch hinzu. Sie können dies mit dem end-Parameter ändern:

```
print("Hallo,", end=" ")  
print("Welt!")  
# Ausgabe: Hallo, Welt!
```

- Sie können mit dem file-Parameter auch in Dateien oder andere Streams schreiben:

```
with open("output.txt", "w") as f:  
    print("Hallo, Datei!", file=f)
```

- `input`: Liest eine Zeile Benutzereingabe von der Konsole und gibt sie als String zurück. Sie können sie bei Bedarf in einen anderen Typ umwandeln.

```
name = input("Wie heißt du? ")  
print("Hallo,", name)
```

```
age = int(input("Wie alt bist du? "))  
print("Dein Alter ist", age)  
print("Du hast", 65 - age, "Jahre bis zur Rente")
```

- `input`: Liest eine Zeile Benutzereingabe von der Konsole und gibt sie als String zurück. Sie können sie bei Bedarf in einen anderen Typ umwandeln.
- Der Prompt-String (falls angegeben) wird vor der Eingabe angezeigt.

```
name = input("Wie heißt du? ")  
print("Hallo,", name)
```

```
age = int(input("Wie alt bist du? "))  
print("Dein Alter ist", age)  
print("Du hast", 65 - age, "Jahre bis zur Rente")
```


- `input()`: Liest eine Zeile Benutzereingabe von der Konsole und gibt sie als String zurück. Sie können sie bei Bedarf in einen anderen Typ umwandeln.
- Der Prompt-String (falls angegeben) wird vor der Eingabe angezeigt.
- Verwenden Sie `int()`, `float()` oder andere Typumwandlungsfunktionen, um numerische Eingaben zu verarbeiten.

```
name = input("Wie heißt du? ")  
print("Hallo,", name)
```

```
age = int(input("Wie alt bist du? "))  
print("Dein Alter ist", age)  
print("Du hast", 65 - age, "Jahre bis zur Rente")
```

- for-Schleife: Wiederholt eine Anweisungsgruppe über eine Gruppe von Werten.

```
for x in range(1, 6):  
    print(x, "zum Quadrat ist", x * x)
```

- for-Schleife: Wiederholt eine Anweisungsgruppe über eine Gruppe von Werten.
- Die for-Schleife in Python wird verwendet, um über Sequenzen (wie Listen, Tupel, Strings), Bereiche oder beliebige iterierbare Objekte zu iterieren.

```
for x in range(1, 6):  
    print(x, "zum Quadrat ist", x * x)
```

- for-Schleife: Wiederholt eine Anweisungsgruppe über eine Gruppe von Werten.
- Die for-Schleife in Python wird verwendet, um über Sequenzen (wie Listen, Tupel, Strings), Bereiche oder beliebige iterierbare Objekte zu iterieren.
- Syntax: `for variable in sequence: statements`

```
for x in range(1, 6):  
    print(x, "zum Quadrat ist", x * x)
```

- for-Schleife: Wiederholt eine Anweisungsgruppe über eine Gruppe von Werten.
- Die for-Schleife in Python wird verwendet, um über Sequenzen (wie Listen, Tupel, Strings), Bereiche oder beliebige iterierbare Objekte zu iterieren.
- Syntax: `for variable in sequence: statements`
- Die Variable nimmt nacheinander jeden Wert aus der Sequenz an, und der eingerückte Block wird für jeden Wert ausgeführt.

```
for x in range(1, 6):  
    print(x, "zum Quadrat ist", x * x)
```

- Sie können for-Schleifen verwenden, um über Listen, Tupel, Strings, Dictionaries und mehr zu iterieren.

```
# Über eine Liste iterieren
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

```
# Über einen String iterieren
for char in "hello":
    print(char)
```

- Die `range()`-Funktion wird häufig verwendet, um eine Zahlenfolge für die Schleife zu erzeugen.

```
# Rückwärtszählen mit Schrittweite  
for i in range(10, 0, -2):  
    print(i)  
# Ausgabe: 10 8 6 4 2
```

- Die `range()`-Funktion wird häufig verwendet, um eine Zahlenfolge für die Schleife zu erzeugen.
- `range(start, stop[, step])` erzeugt Zahlen von `start` (einschließlich) bis `stop` (ausschließlich), optional mit Schrittweite.

```
# Rückwärtszählen mit Schrittweite
for i in range(10, 0, -2):
    print(i)
# Ausgabe: 10 8 6 4 2
```


- Verwenden Sie `enumerate()`-Funktion, um beim Durchlaufen einer Sequenz sowohl den Index als auch den Wert zu erhalten.

```
colors = ["red", "green", "blue"]  
for idx, color in enumerate(colors):  
    print(idx, color)
```

```
# Ausgabe:
```

```
# 0 red
```

```
# 1 green
```

```
# 2 blue
```

- Mit `break` können Sie die Schleife vorzeitig verlassen, mit `continue` zur nächsten Iteration springen.

```
for n in range(1, 10):  
    if n == 5:  
        break  
    print(n)  
# Ausgabe: 1 2 3 4
```

```
for n in range(1, 6):  
    if n == 3:  
        continue  
    print(n)  
# Ausgabe: 1 2 4 5
```

- Sie können else mit einer for-Schleife verwenden; der else-Block wird ausgeführt, wenn die Schleife ohne break endet.

```
for n in range(3):  
    print(n)  
else:  
    print("Schleife beendet!")  
# Ausgabe: 0 1 2 Schleife beendet!
```

- Manche Schleifen berechnen schrittweise einen Wert, der außerhalb der Schleife initialisiert wird.

- Manche Schleifen berechnen schrittweise einen Wert, der außerhalb der Schleife initialisiert wird.
- Dies wird manchmal als kumulative Summe oder Akkumulator-Muster bezeichnet.

- Manche Schleifen berechnen schrittweise einen Wert, der außerhalb der Schleife initialisiert wird.
- Dies wird manchmal als kumulative Summe oder Akkumulator-Muster bezeichnet.
- Die Variable, die das Ergebnis aufsummiert (akkumuliert), wird typischerweise vor der Schleife initialisiert und innerhalb der Schleife aktualisiert.

- Manche Schleifen berechnen schrittweise einen Wert, der außerhalb der Schleife initialisiert wird.
- Dies wird manchmal als kumulative Summe oder Akkumulator-Muster bezeichnet.
- Die Variable, die das Ergebnis aufsummiert (akkumuliert), wird typischerweise vor der Schleife initialisiert und innerhalb der Schleife aktualisiert.
- Dieses Muster ist nützlich zum Summieren, Multiplizieren, Zählen oder für andere Aggregationen von Werten.

```
# Summe der Quadrate der ersten 10 Zahlen berechnen
summe = 0
for i in range(1, 11):
    summe = summe + (i * i)
print("Summe der ersten 10 Quadrate ist", summe)
```



```
# Zählen, wie viele gerade Zahlen in einer Liste sind
zahlen = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
anzahl = 0
for n in zahlen:
    if n % 2 == 0:
        anzahl += 1
print("Anzahl der geraden Zahlen:", anzahl)
```

```
# Produkt einer Sequenz berechnen (Fakultät von 5)
produkt = 1
for i in range(1, 6):
    produkt *= i
print("5! =", produkt)
```

- if-Anweisung: Führt eine Gruppe von Anweisungen nur aus, wenn eine Bedingung wahr ist.

- if-Anweisung: Führt eine Gruppe von Anweisungen nur aus, wenn eine Bedingung wahr ist.
- if/else-Anweisung: Führt einen Block aus, wenn die Bedingung wahr ist, andernfalls einen anderen Block.

- if-Anweisung: Führt eine Gruppe von Anweisungen nur aus, wenn eine Bedingung wahr ist.
- if/else-Anweisung: Führt einen Block aus, wenn die Bedingung wahr ist, andernfalls einen anderen Block.
- elif ("else if") ermöglicht es, mehrere Bedingungen der Reihe nach zu prüfen.

- if-Anweisung: Führt eine Gruppe von Anweisungen nur aus, wenn eine Bedingung wahr ist.
- if/else-Anweisung: Führt einen Block aus, wenn die Bedingung wahr ist, andernfalls einen anderen Block.
- elif ("else if") ermöglicht es, mehrere Bedingungen der Reihe nach zu prüfen.
- Die Bedingung in einer if-Anweisung ist ein Ausdruck, der zu True oder False ausgewertet wird.

- if-Anweisung: Führt eine Gruppe von Anweisungen nur aus, wenn eine Bedingung wahr ist.
- if/else-Anweisung: Führt einen Block aus, wenn die Bedingung wahr ist, andernfalls einen anderen Block.
- elif ("else if") ermöglicht es, mehrere Bedingungen der Reihe nach zu prüfen.
- Die Bedingung in einer if-Anweisung ist ein Ausdruck, der zu True oder False ausgewertet wird.
- Die Einrückung wird verwendet, um die Blöcke zu definieren, die zu jedem Zweig gehören.

```
gpa = 1.4
if gpa > 2.0:
    print("Willkommen an der Mars-Universität!")
else:
    print("Ihre Bewerbung wurde abgelehnt.")
```


- Beispiel mit elif:

```
score = 85
if score >= 90:
    print("Note: A")
elif score >= 80:
    print("Note: B")
elif score >= 70:
    print("Note: C")
else:
    print("Note: D oder F")
```

- Sie können jeden Ausdruck verwenden, der zu True oder False ausgewertet wird:

```
name = "Alice"
if name:
    print("Name ist nicht leer")
if len(name) > 3:
    print("Name ist länger als 3 Zeichen")
```

- Verschachtelte if-Anweisungen sind ebenfalls möglich:

```
age = 20
if age >= 18:
    print("Du bist volljährig.")
    if age >= 21:
        print("Du darfst in den USA Alkohol trinken.")
    else:
        print("Du darfst in den USA keinen Alkohol trinken.")
else:
    print("Du bist minderjährig.")
```

- Die pass-Anweisung kann als Platzhalter für einen leeren Block verwendet werden:

```
if False:
    pass # Nichts tun
else:
    print("Dies wird immer ausgegeben.")
```

- Boolesche Ausdrücke können logische Operatoren (and, or, not) verwenden:

```
x = 5
```

```
y = 10
```

```
if x < 10 and y > 5:
```

```
    print("Beide Bedingungen sind wahr")
```

- while-Schleife: Führt eine Gruppe von Anweisungen aus, solange eine Bedingung wahr ist.
- Die while-Schleife ist nützlich, wenn Sie nicht im Voraus wissen, wie oft eine Aktion wiederholt werden muss.
- Die Schleife läuft so lange, wie die Bedingung wahr ist; ist die Bedingung zu Beginn falsch, wird der Schleifenrumpf nie ausgeführt.
- Achten Sie darauf, Variablen in der Schleife so zu aktualisieren, dass die Bedingung irgendwann falsch wird – sonst entsteht eine Endlosschleife.

```
number = 1
while number < 200:
    print(number)
    number = number * 2
```

While-Schleifen

- Beispiel: Benutzereingaben zählen, bis ein Abbruchwert eingegeben wird

```
gesamt = 0
anzahl = 0
print("0 zum Beenden")
num = int(input("Geben Sie eine Zahl ein: "))
while num != 0:
    gesamt += num
    anzahl += 1
    print("0 zum Beenden")
    num = int(input("Geben Sie eine Zahl ein: "))
print("Anzahl:" anzahl)
print("Summe:", gesamt)
```


- Beispiel: Benutzereingabe validieren

```
passwort = ""  
while passwort != "geheim":  
    passwort = input("Geben Sie das Passwort ein: ")  
print("Zugang gewährt!")
```

- Mit `break` kann man eine `while`-Schleife vorzeitig verlassen, mit `continue` zur nächsten Iteration springen.

```
i = 0
while True:
    i += 1
    if i == 5:
        continue # 5 wird übersprungen
    if i > 10:
        break
    print(i)
```

- Der else-Block nach einer while-Schleife wird ausgeführt, wenn die Schleifenbedingung falsch wird (nicht bei break).

```
n = 3
while n > 0:
    print(n)
    n -= 1
else:
    print("Schleife beendet!")
```

- Vergleichsoperatoren: =, !=, <, >, <=, >=
 - == : gleich
 - != : ungleich
 - < : kleiner als
 - > : größer als
 - <= : kleiner oder gleich
 - >= : größer oder gleich
- Logische Operatoren: and, or, not
 - and: True, wenn beide Seiten True sind
 - or: True, wenn mindestens eine Seite True ist
 - not: Negiert den Wahrheitswert

- Diese Operatoren werden verwendet, um boolesche Ausdrücke zu bilden, die für die Steuerung des Programmablaufs mit if, while und anderen Anweisungen unerlässlich sind.

```
x = 5
```

```
y = 8
```

```
# Vergleichsoperatoren
```

```
print(x == 5)    # True
```

```
print(y != 10)   # True
```

```
print(x < y)      # True
```

```
print(x >= 0)     # True
```

```
# Logische Operatoren
```

```
if x > 0 and y < 10:
```

```
    print("x ist positiv und y ist kleiner als 10")
```

- Listen sind geordnete Sammlungen von Daten, veränderbar (können nach der Erstellung geändert werden) und können Elemente unterschiedlichen Typs enthalten.
- Listen sind eine der am häufigsten verwendeten Datenstrukturen in Python.
- Listen werden mit eckigen Klammern `[]` definiert, die Elemente werden durch Kommas getrennt.
- Listen können indiziert, gesliced, durchlaufen (iteriert) und verändert werden.
- Listen können beliebige Objekte enthalten, auch andere Listen (verschachtelte Listen).

```
x = [1, 'hello', (3 + 2j)]  
x[2]      # (3+2j)  
x[0:2]    # [1, 'hello']  
x.append(12) # Fügt 12 am Ende hinzu  
print(x)    # [1, 'hello', (3+2j), 12]
```

- Indizierung und Slicing:
 - Indizierung beginnt bei 0. Negative Indizes zählen vom Ende.
 - Slicing: `list[start:stop]` gibt eine neue Liste von Index start bis, aber ohne, stop zurück.

```
zahlen = [10, 20, 30, 40, 50]
print(zahlen[0])      # 10
print(zahlen[-1])     # 50
print(zahlen[1:4])    # [20, 30, 40]
print(zahlen[:3])     # [10, 20, 30]
print(zahlen[3:])     # [40, 50]
```


- Listen sind veränderbar (mutable):
 - Sie können Elemente per Zuweisung ändern.
 - Sie können Elemente hinzufügen, entfernen oder verändern.

```
fruechte = ["apple", "banana", "cherry"]
fruechte[1] = "blueberry"    # Element ändern
fruechte.append("date")      # Am Ende hinzufügen
fruechte.insert(1, "kiwi")    # An Index 1 einfügen
print(fruechte)              # ['apple', 'kiwi', 'blueberry']
fruechte.remove("kiwi")      # Nach Wert entfernen
item = fruechte.pop()        # Letztes entfernen und zurückgeben
print(item)                  # 'date'
print(fruechte)              # ['apple', 'blueberry', 'cherry']
```

- Wichtige Listenfunktionen und -methoden:

- Wichtige Listenfunktionen und -methoden:
 - `len(liste)`: Anzahl der Elemente

- Wichtige Listenfunktionen und -methoden:
 - `len(liste)`: Anzahl der Elemente
 - `liste.append(x)`: Fügt `x` am Ende hinzu

- Wichtige Listenfunktionen und -methoden:
 - `len(liste)`: Anzahl der Elemente
 - `liste.append(x)`: Fügt `x` am Ende hinzu
 - `liste.insert(i, x)`: Fügt `x` an Index `i` ein

- Wichtige Listenfunktionen und -methoden:
 - `len(liste)`: Anzahl der Elemente
 - `liste.append(x)`: Fügt `x` am Ende hinzu
 - `liste.insert(i, x)`: Fügt `x` an Index `i` ein
 - `liste.remove(x)`: Entfernt das erste Vorkommen von `x`

- Wichtige Listenfunktionen und -methoden:
 - `len(liste)`: Anzahl der Elemente
 - `liste.append(x)`: Fügt `x` am Ende hinzu
 - `liste.insert(i, x)`: Fügt `x` an Index `i` ein
 - `liste.remove(x)`: Entfernt das erste Vorkommen von `x`
 - `liste.pop([i])`: Entfernt und gibt das Element an Index `i` zurück (Standard: letztes)

- Wichtige Listenfunktionen und -methoden:
 - `len(liste)`: Anzahl der Elemente
 - `liste.append(x)`: Fügt `x` am Ende hinzu
 - `liste.insert(i, x)`: Fügt `x` an Index `i` ein
 - `liste.remove(x)`: Entfernt das erste Vorkommen von `x`
 - `liste.pop([i])`: Entfernt und gibt das Element an Index `i` zurück (Standard: letztes)
 - `liste.index(x)`: Gibt den Index des ersten Vorkommens von `x` zurück

- Wichtige Listenfunktionen und -methoden:
 - `len(liste)`: Anzahl der Elemente
 - `liste.append(x)`: Fügt `x` am Ende hinzu
 - `liste.insert(i, x)`: Fügt `x` an Index `i` ein
 - `liste.remove(x)`: Entfernt das erste Vorkommen von `x`
 - `liste.pop([i])`: Entfernt und gibt das Element an Index `i` zurück (Standard: letztes)
 - `liste.index(x)`: Gibt den Index des ersten Vorkommens von `x` zurück
 - `liste.count(x)`: Zählt Vorkommen von `x`

- Wichtige Listenfunktionen und -methoden:
 - `len(liste)`: Anzahl der Elemente
 - `liste.append(x)`: Fügt `x` am Ende hinzu
 - `liste.insert(i, x)`: Fügt `x` an Index `i` ein
 - `liste.remove(x)`: Entfernt das erste Vorkommen von `x`
 - `liste.pop([i])`: Entfernt und gibt das Element an Index `i` zurück (Standard: letztes)
 - `liste.index(x)`: Gibt den Index des ersten Vorkommens von `x` zurück
 - `liste.count(x)`: Zählt Vorkommen von `x`
 - `liste.sort()`: Sortiert die Liste in-place

- Wichtige Listenfunktionen und -methoden:
 - `len(liste)`: Anzahl der Elemente
 - `liste.append(x)`: Fügt `x` am Ende hinzu
 - `liste.insert(i, x)`: Fügt `x` an Index `i` ein
 - `liste.remove(x)`: Entfernt das erste Vorkommen von `x`
 - `liste.pop([i])`: Entfernt und gibt das Element an Index `i` zurück (Standard: letztes)
 - `liste.index(x)`: Gibt den Index des ersten Vorkommens von `x` zurück
 - `liste.count(x)`: Zählt Vorkommen von `x`
 - `liste.sort()`: Sortiert die Liste in-place
 - `liste.reverse()`: Kehrt die Reihenfolge um

- Wichtige Listenfunktionen und -methoden:
 - `len(liste)`: Anzahl der Elemente
 - `liste.append(x)`: Fügt `x` am Ende hinzu
 - `liste.insert(i, x)`: Fügt `x` an Index `i` ein
 - `liste.remove(x)`: Entfernt das erste Vorkommen von `x`
 - `liste.pop([i])`: Entfernt und gibt das Element an Index `i` zurück (Standard: letztes)
 - `liste.index(x)`: Gibt den Index des ersten Vorkommens von `x` zurück
 - `liste.count(x)`: Zählt Vorkommen von `x`
 - `liste.sort()`: Sortiert die Liste in-place
 - `liste.reverse()`: Kehrt die Reihenfolge um
 - `liste.copy()`: Gibt eine flache Kopie zurück

- Wichtige Listenfunktionen und -methoden:
 - `len(liste)`: Anzahl der Elemente
 - `liste.append(x)`: Fügt `x` am Ende hinzu
 - `liste.insert(i, x)`: Fügt `x` an Index `i` ein
 - `liste.remove(x)`: Entfernt das erste Vorkommen von `x`
 - `liste.pop([i])`: Entfernt und gibt das Element an Index `i` zurück (Standard: letztes)
 - `liste.index(x)`: Gibt den Index des ersten Vorkommens von `x` zurück
 - `liste.count(x)`: Zählt Vorkommen von `x`
 - `liste.sort()`: Sortiert die Liste in-place
 - `liste.reverse()`: Kehrt die Reihenfolge um
 - `liste.copy()`: Gibt eine flache Kopie zurück
 - `liste.clear()`: Entfernt alle Elemente

```
nums = [3, 1, 4, 1, 5, 9, 2]
print(len(nums))      # 7
print(nums.count(1))  # 2
nums.sort()
print(nums)  # [1, 1, 2, 3, 4, 5, 9]
nums.reverse()
print(nums)  # [9, 5, 4, 3, 2, 1, 1]
```

- Listen-Komprehension: kompakte Möglichkeit, Listen zu erzeugen

```
quadrade = [x * x for x in range(1, 6)]  
print(quadrade)  # [1, 4, 9, 16, 25]
```

```
gerade = [n for n in range(10) if n % 2 == 0]  
print(gerade)    # [0, 2, 4, 6, 8]
```

- Verschachtelte Listen (Listen von Listen):

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
print(matrix[1][2])  # 6
```


- Listen kopieren:
 - Mit `liste.copy()` oder `list()` für eine flache Kopie.
 - Mit `copy.deepcopy()` für eine tiefe Kopie (bei verschachtelten Listen).

```
import copy
a = [[1, 2], [3, 4]]
b = a.copy()           # Flache Kopie
c = copy.deepcopy(a)   # Tiefe Kopie
a[0][0] = 99
print(b)  # [[99, 2], [3, 4]]
print(c)  # [[1, 2], [3, 4]]
```

Das Wichtigste:

- Listen sind dynamisch: Sie können beliebig wachsen oder schrumpfen.
- Listen können beliebige Objekte enthalten, auch andere Listen, Funktionen oder sogar Klassen.

- Tupel sind unveränderliche (immutable) Versionen von Listen.
- Nach der Erstellung können die Elemente eines Tupels nicht mehr geändert, hinzugefügt oder entfernt werden.
- Tupel werden mit runden Klammern () definiert, die Elemente werden durch Kommas getrennt.
- Tupel können Elemente unterschiedlichen Typs enthalten, auch andere Tupel oder Listen.
- Tupel werden oft für feste Sammlungen von Werten verwendet, z.B. Koordinaten, RGB-Farben oder Rückgabewerte von Funktionen.

- Sie können Tupel für das Unpacking (mehrfache Zuweisung), als Schlüssel in Dictionaries und in Sets verwenden (da sie hashbar sind, wenn alle Elemente hashbar sind).
- Ein Tupel mit nur einem Element benötigt ein Komma am Ende, z.B. (2,).

```
x = (1, 2, 3)
y = (2,)
z = ("apple", 3.14, [1, 2, 3])
print(x[1])      # 2
print(x[:2])     # (1, 2)
# Tuple Unpacking
a, b, c = x
print(a, b, c)   # 1 2 3
# Verschachteltes Tupel
t = (1, (2, 3), 4)
print(t[1][0])   # 2
```

- Tupel sind unveränderlich, daher führt Folgendes zu einem Fehler:

```
x = (1, 2, 3)
# x[0] = 10
# TypeError: 'tuple' object does not support
# item assignment
```

- Tupel können als Schlüssel in Dictionaries verwendet werden, im Gegensatz zu Listen:

```
orte = { (52.5, 13.4): "Berlin", (48.8, 2.3): "Paris" }
print(orte[(52.5, 13.4)]) # Berlin
```

- Funktionen können mehrere Werte als Tupel zurückgeben:

```
def min_max(zahlen):  
    return min(zahlen), max(zahlen)
```

```
lo, hi = min_max([1, 2, 3, 4])  
print(lo, hi)  # 1 4
```

- Sie können zwischen Listen und Tupeln konvertieren:

```
lst = [1, 2, 3]  
tup = tuple(lst)  
lst2 = list(tup)
```

- Sets (Mengen) sind ungeordnete Sammlungen von einzigartigen Elementen (keine Duplikate).

Mengen (Sets)

- Sets (Mengen) sind ungeordnete Sammlungen von einzigartigen Elementen (keine Duplikate).
- Sets sind veränderbar: Sie können nach der Erstellung Elemente hinzufügen oder entfernen.

Mengen (Sets)

- Sets (Mengen) sind ungeordnete Sammlungen von einzigartigen Elementen (keine Duplikate).
- Sets sind veränderbar: Sie können nach der Erstellung Elemente hinzufügen oder entfernen.
- Sets werden mit dem `set()`-Konstruktor oder geschweiften Klammern `{}` definiert (außer für leere Sets, hier muss `set()` verwendet werden).

Mengen (Sets)

- Sets (Mengen) sind ungeordnete Sammlungen von einzigartigen Elementen (keine Duplikate).
- Sets sind veränderbar: Sie können nach der Erstellung Elemente hinzufügen oder entfernen.
- Sets werden mit dem `set()`-Konstruktor oder geschweiften Klammern `{}` definiert (außer für leere Sets, hier muss `set()` verwendet werden).
- Sets sind nützlich für Mitgliedschaftsprüfungen, das Entfernen von Duplikaten aus Sequenzen und für mathematische Mengenoperationen.

Mengen (Sets)

- Sets (Mengen) sind ungeordnete Sammlungen von einzigartigen Elementen (keine Duplikate).
- Sets sind veränderbar: Sie können nach der Erstellung Elemente hinzufügen oder entfernen.
- Sets werden mit dem `set()`-Konstruktor oder geschweiften Klammern `{}` definiert (außer für leere Sets, hier muss `set()` verwendet werden).
- Sets sind nützlich für Mitgliedschaftsprüfungen, das Entfernen von Duplikaten aus Sequenzen und für mathematische Mengenoperationen.
- Elemente eines Sets müssen unveränderlich (hashbar) sein, z.B. Zahlen, Strings, Tupel (keine Listen oder anderen Sets).

```
setA = set(["a", "b", "c", "d"])
setB = set(["c", "d", "e", "f"])
setC = {"x", "y", "z"} # Set-Literal
leeres_set = set()
# NICHT {} (das ist ein leeres Dictionary)
print("a" in setA)      # True
print("e" in setA)      # False
```

- Häufige Mengenoperationen:

- Häufige Mengenoperationen:
 - $\text{setA} - \text{setB}$: Differenz (Elemente in setA , aber nicht in setB)

- Häufige Mengenoperationen:
 - $\text{setA} - \text{setB}$: Differenz (Elemente in setA, aber nicht in setB)
 - $\text{setA} \mid \text{setB}$: Vereinigung (alle Elemente aus beiden Sets)

- Häufige Mengenoperationen:
 - $\text{setA} - \text{setB}$: Differenz (Elemente in setA, aber nicht in setB)
 - $\text{setA} \mid \text{setB}$: Vereinigung (alle Elemente aus beiden Sets)
 - $\text{setA} \& \text{setB}$: Schnittmenge (Elemente in beiden Sets)

- Häufige Mengenoperationen:
 - $\text{setA} - \text{setB}$: Differenz (Elemente in setA, aber nicht in setB)
 - $\text{setA} \cup \text{setB}$: Vereinigung (alle Elemente aus beiden Sets)
 - $\text{setA} \cap \text{setB}$: Schnittmenge (Elemente in beiden Sets)
 - $\text{setA} \Delta \text{setB}$: Symmetrische Differenz (Elemente in setA oder setB, aber nicht in beiden)

- Häufige Mengenoperationen:
 - $\text{setA} - \text{setB}$: Differenz (Elemente in setA , aber nicht in setB)
 - $\text{setA} \mid \text{setB}$: Vereinigung (alle Elemente aus beiden Sets)
 - $\text{setA} \& \text{setB}$: Schnittmenge (Elemente in beiden Sets)
 - $\text{setA} \hat{\ } \text{setB}$: Symmetrische Differenz (Elemente in setA oder setB , aber nicht in beiden)
 - $\text{setA.add}(x)$: Element x hinzufügen

- Häufige Mengenoperationen:
 - $\text{setA} - \text{setB}$: Differenz (Elemente in setA, aber nicht in setB)
 - $\text{setA} \cup \text{setB}$: Vereinigung (alle Elemente aus beiden Sets)
 - $\text{setA} \cap \text{setB}$: Schnittmenge (Elemente in beiden Sets)
 - $\text{setA} \Delta \text{setB}$: Symmetrische Differenz (Elemente in setA oder setB, aber nicht in beiden)
 - `setA.add(x)`: Element x hinzufügen
 - `setA.remove(x)`: Element x entfernen (Fehler, falls nicht vorhanden)

- Häufige Mengenoperationen:
 - `setA - setB`: Differenz (Elemente in `setA`, aber nicht in `setB`)
 - `setA | setB`: Vereinigung (alle Elemente aus beiden Sets)
 - `setA & setB`: Schnittmenge (Elemente in beiden Sets)
 - `setA ^ setB`: Symmetrische Differenz (Elemente in `setA` oder `setB`, aber nicht in beiden)
 - `setA.add(x)`: Element `x` hinzufügen
 - `setA.remove(x)`: Element `x` entfernen (Fehler, falls nicht vorhanden)
 - `setA.discard(x)`: Element `x` entfernen, falls vorhanden (kein Fehler, falls nicht)

- Häufige Mengenoperationen:
 - `setA - setB`: Differenz (Elemente in `setA`, aber nicht in `setB`)
 - `setA | setB`: Vereinigung (alle Elemente aus beiden Sets)
 - `setA & setB`: Schnittmenge (Elemente in beiden Sets)
 - `setA ^ setB`: Symmetrische Differenz (Elemente in `setA` oder `setB`, aber nicht in beiden)
 - `setA.add(x)`: Element `x` hinzufügen
 - `setA.remove(x)`: Element `x` entfernen (Fehler, falls nicht vorhanden)
 - `setA.discard(x)`: Element `x` entfernen, falls vorhanden (kein Fehler, falls nicht)
 - `setA.pop()`: Entfernt und gibt ein beliebiges Element zurück

- Häufige Mengenoperationen:
 - `setA - setB`: Differenz (Elemente in `setA`, aber nicht in `setB`)
 - `setA | setB`: Vereinigung (alle Elemente aus beiden Sets)
 - `setA & setB`: Schnittmenge (Elemente in beiden Sets)
 - `setA ^ setB`: Symmetrische Differenz (Elemente in `setA` oder `setB`, aber nicht in beiden)
 - `setA.add(x)`: Element `x` hinzufügen
 - `setA.remove(x)`: Element `x` entfernen (Fehler, falls nicht vorhanden)
 - `setA.discard(x)`: Element `x` entfernen, falls vorhanden (kein Fehler, falls nicht)
 - `setA.pop()`: Entfernt und gibt ein beliebiges Element zurück
 - `setA.clear()`: Entfernt alle Elemente

Mengen (Sets)

```
setA = set(["a", "b", "c", "d"])
setB = set(["c", "d", "e", "f"])
print(setA - setB)  # {'a', 'b'}
print(setA | setB)  # {'a', 'b', 'c', 'd', 'e', 'f'}
print(setA & setB)  # {'c', 'd'}
print(setA ^ setB)  # {'a', 'b', 'e', 'f'}

setA.add("z")
setA.remove("a")
setA.discard("not_in_set")  # Kein Fehler
print(setA)
```


- Sets sind iterierbar und können in for-Schleifen verwendet werden:

```
for item in setA:  
    print(item)
```

- Sie können Listen in Sets umwandeln, um Duplikate zu entfernen:

```
lst = [1, 2, 2, 3, 4, 4, 5]  
unique = set(lst)  
print(unique)  # {1, 2, 3, 4, 5}
```

- Sets unterstützen kein Indizieren oder Slicing, da sie ungeordnet sind.
- Frozenset ist eine unveränderliche Version eines Sets:

```
fs = frozenset([1, 2, 3])  
# fs.add(4)  
# AttributeError: 'frozenset' object has no attribute 'add'
```

- Dictionaries sind veränderbare Sammlungen von Schlüssel-Wert-Paaren.

- Dictionaries sind veränderbare Sammlungen von Schlüssel-Wert-Paaren.
- Schlüssel müssen eindeutig und unveränderlich (z.B. Strings, Zahlen, Tupel) sein; Werte können beliebigen Typs sein.

- Dictionaries sind veränderbare Sammlungen von Schlüssel-Wert-Paaren.
- Schlüssel müssen eindeutig und unveränderlich (z.B. Strings, Zahlen, Tupel) sein; Werte können beliebigen Typs sein.
- Dictionaries werden mit geschweiften Klammern `{}` und Schlüssel:Wert-Paaren oder mit dem `dict()`-Konstruktor definiert.

- Dictionaries sind veränderbare Sammlungen von Schlüssel-Wert-Paaren.
- Schlüssel müssen eindeutig und unveränderlich (z.B. Strings, Zahlen, Tupel) sein; Werte können beliebigen Typs sein.
- Dictionaries werden mit geschweiften Klammern `{}` und Schlüssel:Wert-Paaren oder mit dem `dict()`-Konstruktor definiert.
- Dictionaries werden häufig für schnelle Nachschlageoperationen, Konfigurationen, Zählungen und vieles mehr verwendet.

```
d = {'one': 1, 'two': 2, 'three': 3}
print(d['three']) # 3
d['two'] = 99      # Wert für Schlüssel 'two' ändern
d['four'] = 4      # Neues Schlüssel-Wert-Paar hinzufügen
print(d)
# {'one': 1, 'two': 99, 'three': 3, 'four': 4}
```


- Sie können jeden unveränderlichen Typ als Schlüssel verwenden (z.B. Zahlen, Strings, Tupel).
- Der Zugriff auf einen nicht vorhandenen Schlüssel löst einen `KeyError` aus. Verwenden Sie `get()`, um dies zu vermeiden:

```
print(d.get('five', 'nicht gefunden')) # nicht gefunden
```

- Elemente mit `del` oder `pop()` entfernen:

```
del d['one']  
value = d.pop('three')  
print(d)      # {'two': 99, 'four': 4}  
print(value)  # 3
```

- Prüfen, ob ein Schlüssel existiert, mit in:

```
if 'two' in d:  
    print("Schlüssel 'two' existiert!")
```

- Über Schlüssel, Werte oder Paare iterieren:

```
for key in d:  
    print(key, d[key])
```

```
for value in d.values():  
    print(value)
```

```
for key, value in d.items():  
    print(key, value)
```

- Dictionary Comprehensions:

```
quadrade = {x: x*x for x in range(5)}  
print(quadrade)  # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

- Verschachtelte Dictionaries:

```
person = {  
    "name": "Alice",  
    "age": 30,  
    "address": {"city": "Berlin", "zip": "10115"}  
}  
print(person["address"]["city"])  # Berlin
```

- Dictionaries kopieren:

```
d2 = d.copy()    # Flache Kopie  
import copy  
d3 = copy.deepcopy(d)  # Tiefe Kopie
```

- Nützliche Methoden: `keys()`, `values()`, `items()`, `get()`, `pop()`, `update()`, `clear()`, `copy()`

- Funktionen werden mit dem Schlüsselwort 'def' definiert, können Parameter und Rückgabewerte haben.

- Funktionen werden mit dem Schlüsselwort 'def' definiert, können Parameter und Rückgabewerte haben.
- Funktionen helfen, Code zu strukturieren, Wiederholungen zu vermeiden und Programme lesbarer und wartbarer zu machen.

- Funktionen werden mit dem Schlüsselwort 'def' definiert, können Parameter und Rückgabewerte haben.
- Funktionen helfen, Code zu strukturieren, Wiederholungen zu vermeiden und Programme lesbarer und wartbarer zu machen.
- Der Funktionsrumpf ist eingerückt, und die Funktion kann mit dem Schlüsselwort 'return' einen Wert zurückgeben.

- Funktionen werden mit dem Schlüsselwort 'def' definiert, können Parameter und Rückgabewerte haben.
- Funktionen helfen, Code zu strukturieren, Wiederholungen zu vermeiden und Programme lesbarer und wartbarer zu machen.
- Der Funktionsrumpf ist eingerückt, und die Funktion kann mit dem Schlüsselwort 'return' einen Wert zurückgeben.
- Parameter sind Variablen, die in der Funktionsdefinition stehen; Argumente sind die Werte, die Sie an die Funktion übergeben.

- Funktionen werden mit dem Schlüsselwort 'def' definiert, können Parameter und Rückgabewerte haben.
- Funktionen helfen, Code zu strukturieren, Wiederholungen zu vermeiden und Programme lesbarer und wartbarer zu machen.
- Der Funktionsrumpf ist eingerückt, und die Funktion kann mit dem Schlüsselwort 'return' einen Wert zurückgeben.
- Parameter sind Variablen, die in der Funktionsdefinition stehen; Argumente sind die Werte, die Sie an die Funktion übergeben.
- Funktionen können Standardwerte für Parameter, variable Argumentlisten und Schlüsselwortargumente haben.

- Funktionen werden mit dem Schlüsselwort 'def' definiert, können Parameter und Rückgabewerte haben.
- Funktionen helfen, Code zu strukturieren, Wiederholungen zu vermeiden und Programme lesbarer und wartbarer zu machen.
- Der Funktionsrumpf ist eingerückt, und die Funktion kann mit dem Schlüsselwort 'return' einen Wert zurückgeben.
- Parameter sind Variablen, die in der Funktionsdefinition stehen; Argumente sind die Werte, die Sie an die Funktion übergeben.
- Funktionen können Standardwerte für Parameter, variable Argumentlisten und Schlüsselwortargumente haben.
- Funktionen können mehrere Werte als Tupel zurückgeben.

- Funktionen werden mit dem Schlüsselwort 'def' definiert, können Parameter und Rückgabewerte haben.
- Funktionen helfen, Code zu strukturieren, Wiederholungen zu vermeiden und Programme lesbarer und wartbarer zu machen.
- Der Funktionsrumpf ist eingerückt, und die Funktion kann mit dem Schlüsselwort 'return' einen Wert zurückgeben.
- Parameter sind Variablen, die in der Funktionsdefinition stehen; Argumente sind die Werte, die Sie an die Funktion übergeben.
- Funktionen können Standardwerte für Parameter, variable Argumentlisten und Schlüsselwortargumente haben.
- Funktionen können mehrere Werte als Tupel zurückgeben.
- Docstrings (dreifach-quotierte Strings) können zur Dokumentation von Funktionen verwendet werden.

```
def maximum(x, y):  
    """Gibt das Maximum von x und y zurück."""  
    if x < y:  
        return y  
    else:  
        return x
```

Funktion mit Standardparameter

```
def begruessung(name="Welt"):  
    print(f"Hallo, {name}!")
```

```
# Funktion, die mehrere Werte zurückgibt
def min_max(zahlen):
    """Gibt das Minimum und Maximum einer Liste zurück."""
    return min(zahlen), max(zahlen)

# Funktion mit variabler Argumentanzahl
def addiere_alle(*args):
    """Gibt die Summe aller Argumente zurück."""
    return sum(args)
```



```
# Funktion mit Schlüsselwortargumenten
def info_ausgeben(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
```

- Funktionen sind First-Class-Objekte: Sie können Variablen zugewiesen, als Argumente übergeben und von anderen Funktionen zurückgegeben werden.
- Lambda-Ausdrücke bieten eine Möglichkeit, kleine anonyme Funktionen zu erstellen:

```
quadrat = lambda x: x * x  
print(quadrat(5))  # 25
```

Höhere Funktionen (Higher-Order Functions)

- Höhere Funktionen sind Funktionen, die andere Funktionen als Argumente akzeptieren oder als Ergebnis zurückgeben.

Höhere Funktionen (Higher-Order Functions)

- Höhere Funktionen sind Funktionen, die andere Funktionen als Argumente akzeptieren oder als Ergebnis zurückgeben.
- Python stellt mehrere eingebaute höhere Funktionen bereit, darunter map, filter und reduce.

Höhere Funktionen (Higher-Order Functions)

- Höhere Funktionen sind Funktionen, die andere Funktionen als Argumente akzeptieren oder als Ergebnis zurückgeben.
- Python stellt mehrere eingebaute höhere Funktionen bereit, darunter map, filter und reduce.
- Diese Funktionen sind nützlich, um eine Funktion auf eine Sequenz anzuwenden, Elemente zu filtern oder Werte zu kombinieren.

Höhere Funktionen (Higher-Order Functions)

- Höhere Funktionen sind Funktionen, die andere Funktionen als Argumente akzeptieren oder als Ergebnis zurückgeben.
- Python stellt mehrere eingebaute höhere Funktionen bereit, darunter `map`, `filter` und `reduce`.
- Diese Funktionen sind nützlich, um eine Funktion auf eine Sequenz anzuwenden, Elemente zu filtern oder Werte zu kombinieren.
- `map(func, iterable)`: Wendet `func` auf jedes Element von `iterable` an und gibt einen Iterator mit den Ergebnissen zurück.

Höhere Funktionen (Higher-Order Functions)

- Höhere Funktionen sind Funktionen, die andere Funktionen als Argumente akzeptieren oder als Ergebnis zurückgeben.
- Python stellt mehrere eingebaute höhere Funktionen bereit, darunter `map`, `filter` und `reduce`.
- Diese Funktionen sind nützlich, um eine Funktion auf eine Sequenz anzuwenden, Elemente zu filtern oder Werte zu kombinieren.
- `map(func, iterable)`: Wendet `func` auf jedes Element von `iterable` an und gibt einen Iterator mit den Ergebnissen zurück.
- `filter(func, iterable)`: Gibt einen Iterator mit den Elementen von `iterable` zurück, für die `func(item)` `True` ergibt.

Höhere Funktionen (Higher-Order Functions)

- Höhere Funktionen sind Funktionen, die andere Funktionen als Argumente akzeptieren oder als Ergebnis zurückgeben.
- Python stellt mehrere eingebaute höhere Funktionen bereit, darunter `map`, `filter` und `reduce`.
- Diese Funktionen sind nützlich, um eine Funktion auf eine Sequenz anzuwenden, Elemente zu filtern oder Werte zu kombinieren.
- `map(func, iterable)`: Wendet `func` auf jedes Element von `iterable` an und gibt einen Iterator mit den Ergebnissen zurück.
- `filter(func, iterable)`: Gibt einen Iterator mit den Elementen von `iterable` zurück, für die `func(item)` `True` ergibt.
- `reduce(func, iterable)`: Wendet `func` kumulativ auf die Elemente von `iterable` an und reduziert sie auf einen einzelnen Wert. (Import aus `functools`.)

Höhere Funktionen (Higher-Order Functions)

```
def verdoppeln(x):  
    return 2 * x  
  
lst = list(range(10))  
print(list(map(verdoppeln, lst)))  
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

Höhere Funktionen (Higher-Order Functions)

```
# Mit filter gerade Zahlen auswählen
```

```
def ist_gerade(x):
```

```
    return x % 2 == 0
```

```
print(list(filter(ist_gerade, lst)))
```

```
# [0, 2, 4, 6, 8]
```

```
# Mit reduce das Produkt aller Zahlen berechnen
```

```
from functools import reduce
```

```
def multiplizieren(x, y):
```

```
    return x * y
```

```
print(reduce(multiplizieren, [1, 2, 3, 4, 5])) # 120
```

Höhere Funktionen (Higher-Order Functions)

```
# Lambda-Ausdrücke mit höheren Funktionen
print(list(map(lambda x: x ** 2, lst)))
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
print(list(filter(lambda x: x > 5, lst)))
# [6, 7, 8, 9]
print(reduce(lambda x, y: x + y, lst))
# 45
```

- Listen-Komprehension ist oft die “pythonicere” Variante für Mapping und Filterung:

```
quadrade = [x ** 2 for x in lst]
gerade = [x for x in lst if x % 2 == 0]
```

- Eine Klasse definiert, wie Objekte aussehen und welche Funktionen auf ihnen arbeiten können.

- Eine Klasse definiert, wie Objekte aussehen und welche Funktionen auf ihnen arbeiten können.
- Ein Objekt ist eine Instanz einer Klasse mit eigenen Daten und Verhalten.

- Eine Klasse definiert, wie Objekte aussehen und welche Funktionen auf ihnen arbeiten können.
- Ein Objekt ist eine Instanz einer Klasse mit eigenen Daten und Verhalten.
- Der erste Parameter jeder Methode in einer Klasse ist `self`, der auf die Instanz selbst verweist.

- Eine Klasse definiert, wie Objekte aussehen und welche Funktionen auf ihnen arbeiten können.
- Ein Objekt ist eine Instanz einer Klasse mit eigenen Daten und Verhalten.
- Der erste Parameter jeder Methode in einer Klasse ist `self`, der auf die Instanz selbst verweist.
- Klassen können Attribute (Felder) und Methoden (Funktionen innerhalb der Klasse) haben.

- Eine Klasse definiert, wie Objekte aussehen und welche Funktionen auf ihnen arbeiten können.
- Ein Objekt ist eine Instanz einer Klasse mit eigenen Daten und Verhalten.
- Der erste Parameter jeder Methode in einer Klasse ist `self`, der auf die Instanz selbst verweist.
- Klassen können Attribute (Felder) und Methoden (Funktionen innerhalb der Klasse) haben.
- Die Methode `__init__` ist der Konstruktor und wird beim Erzeugen einer neuen Instanz aufgerufen.

- Methoden werden mit Punktnotation aufgerufen:
`objekt.methode(argumente)`.

- Methoden werden mit Punktnotation aufgerufen:
`objekt.methode(argumente)`.
- Die Methode `__str__` definiert, wie das Objekt als String ausgegeben wird.

- Methoden werden mit Punktnotation aufgerufen:
`objekt.methode(argumente)`.
- Die Methode `__str__` definiert, wie das Objekt als String ausgegeben wird.
- Python erzwingt keine Zugriffsbeschränkungen (wie `private/protected` in anderen Sprachen), aber ein führender Unterstrich (z.B. `_x`) ist Konvention für "intern".

- Methoden werden mit Punktnotation aufgerufen:
`objekt.methode(argumente)`.
- Die Methode `__str__` definiert, wie das Objekt als String ausgegeben wird.
- Python erzwingt keine Zugriffsbeschränkungen (wie `private/protected` in anderen Sprachen), aber ein führender Unterstrich (z.B. `_x`) ist Konvention für "intern".
- Alle Klassen erben standardmäßig von `object` (in Python 3).

- Methoden werden mit Punktnotation aufgerufen:
`objekt.methode(argumente)`.
- Die Methode `__str__` definiert, wie das Objekt als String ausgegeben wird.
- Python erzwingt keine Zugriffsbeschränkungen (wie `private/protected` in anderen Sprachen), aber ein führender Unterstrich (z.B. `_x`) ist Konvention für "intern".
- Alle Klassen erben standardmäßig von `object` (in Python 3).
- Mit `type(obj)` kann man den Typ eines Objekts prüfen, mit `isinstance(obj, Klassenname)` die Zugehörigkeit zu einer Klasse.

- Methoden werden mit Punktnotation aufgerufen:
`objekt.methode(argumente)`.
- Die Methode `__str__` definiert, wie das Objekt als String ausgegeben wird.
- Python erzwingt keine Zugriffsbeschränkungen (wie `private/protected` in anderen Sprachen), aber ein führender Unterstrich (z.B. `_x`) ist Konvention für "intern".
- Alle Klassen erben standardmäßig von `object` (in Python 3).
- Mit `type(obj)` kann man den Typ eines Objekts prüfen, mit `isinstance(obj, Klassenname)` die Zugehörigkeit zu einer Klasse.
- Sie können spezielle Methoden definieren, um das Verhalten zu beeinflussen, z.B. `__add__`, `__eq__`, `__len__` usw.

- Sie können auch statische Methoden (mit @staticmethod) definieren, die nicht auf Instanz- oder Klassendaten zugreifen.

```
class MatheUtils:
    @staticmethod
    def mittelpunkt(x1, y1, x2, y2):
        return ((x1 + x2) / 2, (y1 + y2) / 2)

print(MatheUtils.mittelpunkt(0, 0, 4, 4))  # (2.0, 2.0)
```

- Python unterstützt Mehrfachvererbung, aber verwenden Sie sie mit Bedacht.
- Alle Objekte in Python sind dynamisch: Sie können Attribute zur Laufzeit hinzufügen oder entfernen, aber das ist für Kerndaten nicht empfohlen.

- Vererbung ermöglicht es einer Klasse (Subklasse), Attribute und Methoden einer anderen Klasse (Superklasse) zu übernehmen.

Vererbung (Inheritance)

- Vererbung ermöglicht es einer Klasse (Subklasse), Attribute und Methoden einer anderen Klasse (Superklasse) zu übernehmen.
- Dadurch wird Code-Wiederverwendung, Erweiterung und Spezialisierung von Verhalten ermöglicht.

Vererbung (Inheritance)

- Vererbung ermöglicht es einer Klasse (Subklasse), Attribute und Methoden einer anderen Klasse (Superklasse) zu übernehmen.
- Dadurch wird Code-Wiederverwendung, Erweiterung und Spezialisierung von Verhalten ermöglicht.
- Syntax: `class SubKlassenName(SuperKlassenName):`

Vererbung (Inheritance)

- Vererbung ermöglicht es einer Klasse (Subklasse), Attribute und Methoden einer anderen Klasse (Superklasse) zu übernehmen.
- Dadurch wird Code-Wiederverwendung, Erweiterung und Spezialisierung von Verhalten ermöglicht.
- Syntax: `class SubKlassenName(SuperKlassenName):`
- Die Subklasse kann Methoden und Attribute der Superklasse überschreiben oder erweitern.

Vererbung (Inheritance)

- Vererbung ermöglicht es einer Klasse (Subklasse), Attribute und Methoden einer anderen Klasse (Superklasse) zu übernehmen.
- Dadurch wird Code-Wiederverwendung, Erweiterung und Spezialisierung von Verhalten ermöglicht.
- Syntax: `class SubKlassenName(SuperKlassenName):`
- Die Subklasse kann Methoden und Attribute der Superklasse überschreiben oder erweitern.
- Mit `super()` können Methoden der Superklasse aufgerufen werden.

Vererbung (Inheritance)

- Vererbung ermöglicht es einer Klasse (Subklasse), Attribute und Methoden einer anderen Klasse (Superklasse) zu übernehmen.
- Dadurch wird Code-Wiederverwendung, Erweiterung und Spezialisierung von Verhalten ermöglicht.
- Syntax: `class SubKlassenName(SuperKlassenName):`
- Die Subklasse kann Methoden und Attribute der Superklasse überschreiben oder erweitern.
- Mit `super()` können Methoden der Superklasse aufgerufen werden.
- Python unterstützt Mehrfachvererbung: `class C(A, B):`

Sie können mit `issubclass(SubKlasse, SuperKlasse)` prüfen, ob eine Klasse von einer anderen erbt.

- Mit `isinstance(obj, Klassenname)` prüfen Sie, ob ein Objekt eine Instanz einer Klasse oder deren Unterklassen ist.

```
print(isinstance(p3d, Punkt3D))  # True
print(isinstance(p3d, Punkt))    # True
print(issubclass(Punkt3D, Punkt)) # True
```

- Beispiel für Mehrfachvererbung:

```
class A:
    def foo(self):
        print("A.foo")
```

```
class B:
    def bar(self):
        print("B.bar")
```

```
class C(A, B):
    pass
```

- Die Method Resolution Order (MRO) bestimmt, welche Methode bei mehreren Oberklassen aufgerufen wird.
- Mit `Klassenname.mro()` können Sie die Reihenfolge anzeigen.

```
print(C.mro())  
# [<class '__main__.C'>, <class '__main__.A'>,  
# <class '__main__.B'>, <class 'object'>]
```